CopyLane
ERD (개체 관계도)

SKN Final Project 최종 산출물 — 표 31 · 외래키 32 · 뷰 5 (2026-10-21)

팀장 **오한빈**
팀원 **박수진 · 권소라 · 소성민 · 이서은**
작성 **권소라 · 2주차 ERD 원 작성 오한빈**
개정 **2026-10-21 · 기준 커밋 ksr b54246d · alembic 0027**

제출용

목차

1. 읽는 법 3
2. 전체 개요 — 표 31 · 외래키 32 5
3. 원천 · 수집 — 표 5 6
4. 원문 · 청크 · 임베딩 — 표 5 7
5. 사전 · 제재 · 법령 — 표 6 8
6. 골든 · 라벨 · 재현성 — 표 3 9
7. 회원 · 검수 이력 — 표 4 10
8. 생성 · 일괄 작업 — 표 3 11
9. 운영 · 관리 — 표 5 12
10. 외래키 32개 목록 13
11. 뷰 5개 15
12. 근거와 확인 16
부록 — 표별 한 줄 설명 17

1. 읽는 법

> **작성** 권소라(02 데이터베이스 · 저장소 설계 문서의 ERD 를 큰 글씨로 다시 그림 · 2주차 ERD 원 작성 오한빈) · **개정** 2026-10-21 · **기준 커밋** `ksr b54246d`
>
> **판형** A3 가로 · 도면 안 가장 작은 글자 9.8 pt(13 px) · 칸 줄 글자 10.2 pt
>
> **원천** 이 PC 의 PostgreSQL(127.0.0.1:5432 · 개발용 복제본) 카탈로그를 **2026-10-21 17:52 KST 에 SELECT 로만** 읽은 값(`pg_attribute` · `pg_constraint` · `pg_indexes` · `pg_views`)을 그대로 옮겼다. 같은 표 · 칸 · FK 를 `ksr b54246d` 의 `db/schema.sql` (데이터 층) · `app/models.py` (런타임 층)와 한 칸씩 대조해 다른 곳이 없음을 확인했다(§ 12).

이 문서는 02 「데이터베이스 · 저장소 설계 문서」 § 4 의 ERD 두 그림을 **사람이 눈으로 읽을 수 있는 크기**로 나눠 다시 그린 것이다. 표 · 칸 · 관계는 02 문서와 같다. 2주차 ERD(2026-09-10 · 표 25)는 지금 표 31 로 늘었고, 한 장에 다 넣어 글자가 읽히지 않았던 것을 전체 개요 1쪽 + 영역별 상세 7쪽으로 나눴다.

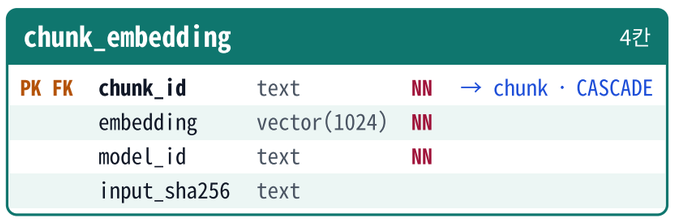
항목	2026-10-21 값	근거
표	**31** = 거버넌스 · 데이터 층 **19**(`db/schema.sql`) + 런타임 층 **12**(`app/models.py`) · 별도로 `alembic_version`	DB `pg_class` · 두 파일의 `CREATE TABLE` · `__tablename__`
칸	**323** (표 31 의 칸 합계)	DB `pg_attribute`
외래키	**32** = CASCADE **25** · SET NULL **2** · NO ACTION **5** — 데이터 층 21 · 런타임 층 11 · **두 층 사이 0**	DB `pg_constraint` (contype f) · `db/schema.sql` REFERENCES 21 · `app/models.py` ForeignKey 11
뷰	**5** — `v_current_chunk` · `v_publishable_golden` · `v_golden_scored` · `v_golden_legal` · `v_risk_lookup`	DB `pg_views` · `db/schema.sql`
열거형	**14** (`grade_t` · `violation_t` 등 `_t` 로 끝나는 타입)	DB `pg_enum`
마이그레이션	head **`0027_judgment_encoder_basis`**	DB `alembic_version` · `alembic/versions/`

쪽 안내 — 영역마다 한 쪽이다. 모든 FK 칸 오른쪽에 「→ 부모 표 · 삭제 규칙」을 적었고(부모 쪽 칸은 언제나 그 표의 기본키), 같은 쪽 안의 관계는 선으로도 이었다 — 선은 부모의 기본키 줄에서 나와 자식의 FK 줄에 닿는다. 부모가 다른 쪽에 있으면 선 대신 이 글자로 읽는다.

영역을 나눈 기준 — 영역은 한 쪽에 읽히는 크기로 나누려고 이 문서가 정한 묶음이다. FK 가 모이는 뿌리(`source` · `fragment` · `document` · `work_doc` · 계정 표)와 쓰임(검색 · 판정 사전 · 골든 평가 · 회원 이력 · 생성 · 운영)을 따라 묶었다. DB 스키마나 코드에 있는 구분은 두 층(데이터 층 `db/schema.sql` · 런타임 층 `app/models.py`)뿐이고, 영역 일곱은 그 안을 다시 나눈 것이다(02 문서 § 5 의 소절 구분과는 다르다).

절	영역	표
§ 2	전체 개요	31표 이름과 FK 32개 선
§ 3	원천 · 수집	`source` · `source_constraint` · `source_use` · `collect_manifest` · `fragment`
§ 4	원문 · 청크 · 임베딩	`document` · `sentence` · `transform_pair` · `chunk` · `chunk_embedding`
§ 5	사전 · 제재 · 법령	`sanction_rule` · `dict_entry` · `product_fact` · `penalty_rule` · `penal_clause` · `violation_article`
§ 6	골든 · 라벨 · 재현성	`golden_sample` · `segment` · `dataset_manifest`
§ 7	회원 · 검수 이력	`user_account` · `work_doc` · `copy_sentence` · `judgment`
§ 8	생성 · 일괄 작업	`batch_run` · `upload_blob` · `slot_assignment`
§ 9	운영 · 관리	`app_account` · `notice` · `terms` · `ticket` · `app_error_log`
§ 10 · § 11 · § 12 · 부록	목록 · 근거	외래키 32 · 뷰 5 · 확인 방법 · 표별 한 줄 설명

보기 — 실제 표 chunk_embedding(이 PC DB 카탈로그 그대로)



① 머리 — 표 이름 · 오른쪽은 칸 수. 머리 색 = 영역(아래 색표)

② 열쇠 — PK 기본키 · FK 외래키 · UQ 유일 제약/유일 인덱스에 든 칸

③ 칸 이름 · 타입(DB 가 돌려준 그대로 · 아래 줄임말) · NN = NOT NULL

④ 참조 — FK 칸 오른쪽에 「→ 부모 표 · 삭제 규칙」

선 — 부모 쪽 막대(하나) ┤ · 자식 쪽 까마귀발(여럿) <. 선 모양 = ON DELETE 규칙

ON DELETE CASCADE(부모를 지우면 함께 지워짐)

ON DELETE SET NULL(부모를 지우면 칸만 비움)

NO ACTION(참조하는 행이 있으면 부모 삭제가 막힘)

다형 참조(DB 외래키 아님)

영역 색 — 상세 쪽 하나가 영역 하나

원천 · 수집

원문 · 청크 · 임베딩

사전 · 제재 · 법령

골든 · 라벨 · 재현성

회원 · 검수 이력

생성 · 일괄 작업

운영 · 관리

그 밖의 표시

· 점선 상자 — 다른 쪽에 칸 전부가 있는 부모 표(그 쪽에서는 기본키 한 줄만 보인다)

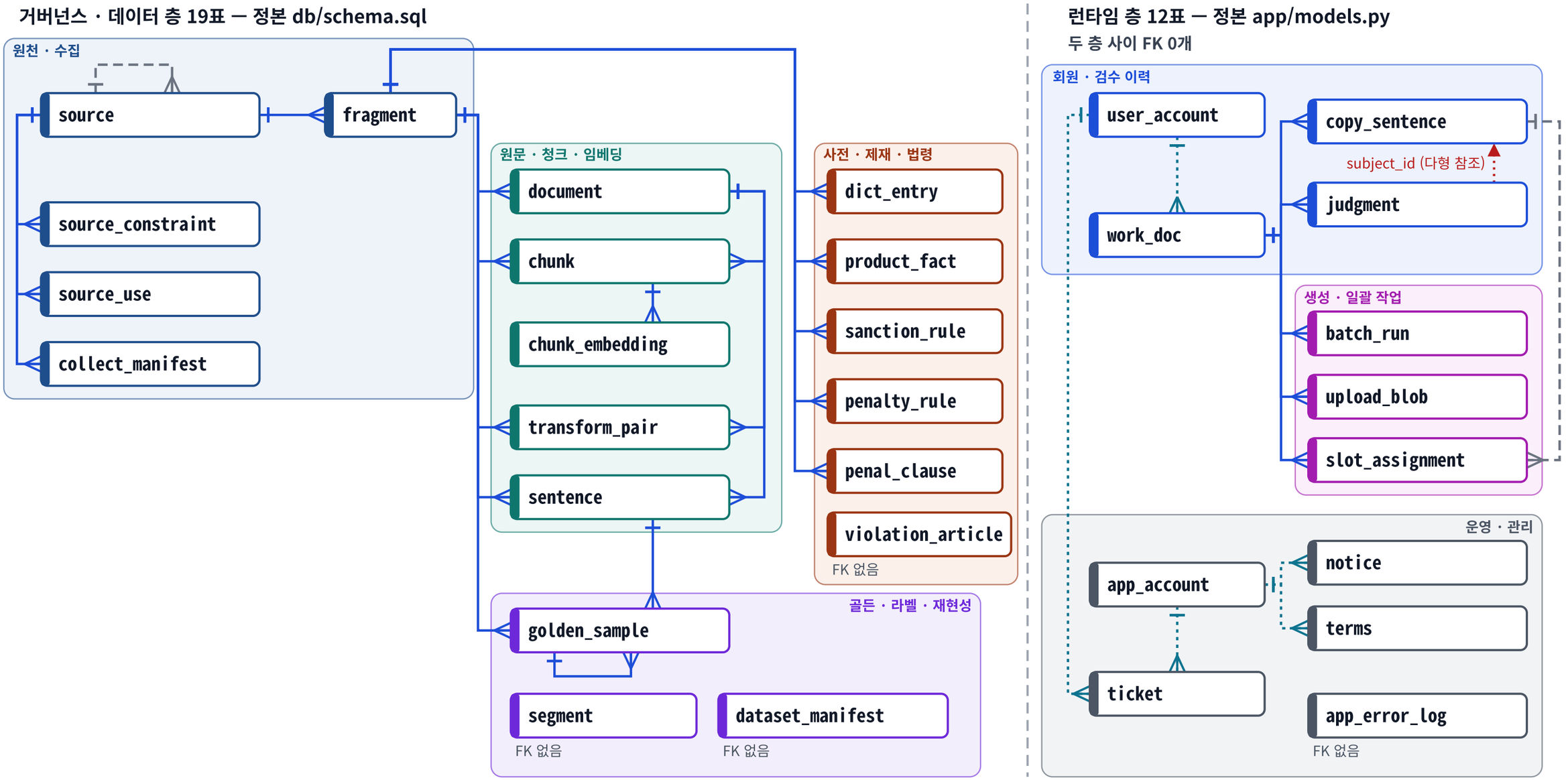
· 줄임말 — varchar(n) = character varying(n) · timestamptz = timestamp with time zone · violation_t 등 _t = 열거형(14종)

· 생성열(GENERATED) — chunk.tsv 는 context · text 로 DB 가 계산해 넣는 칸(to_tsvector simple · GIN 색인 ix_chunk_tsv)

· → 다형 참조 · FK 아님(빨간 글자) — judgment.subject_id 는 subject_type 이 가리키는 표의 id 를 담는다. DB 외래키가 아니다

· 한 표의 여러 칸에 PK 나 UQ 가 붙으면 그 칸들의 묶음이 유일하다(복합 키) — 묶음은 각 상세 쪽 아래 설명에 적었다

2. 전체 개요 — 표 31 · 외래키 32

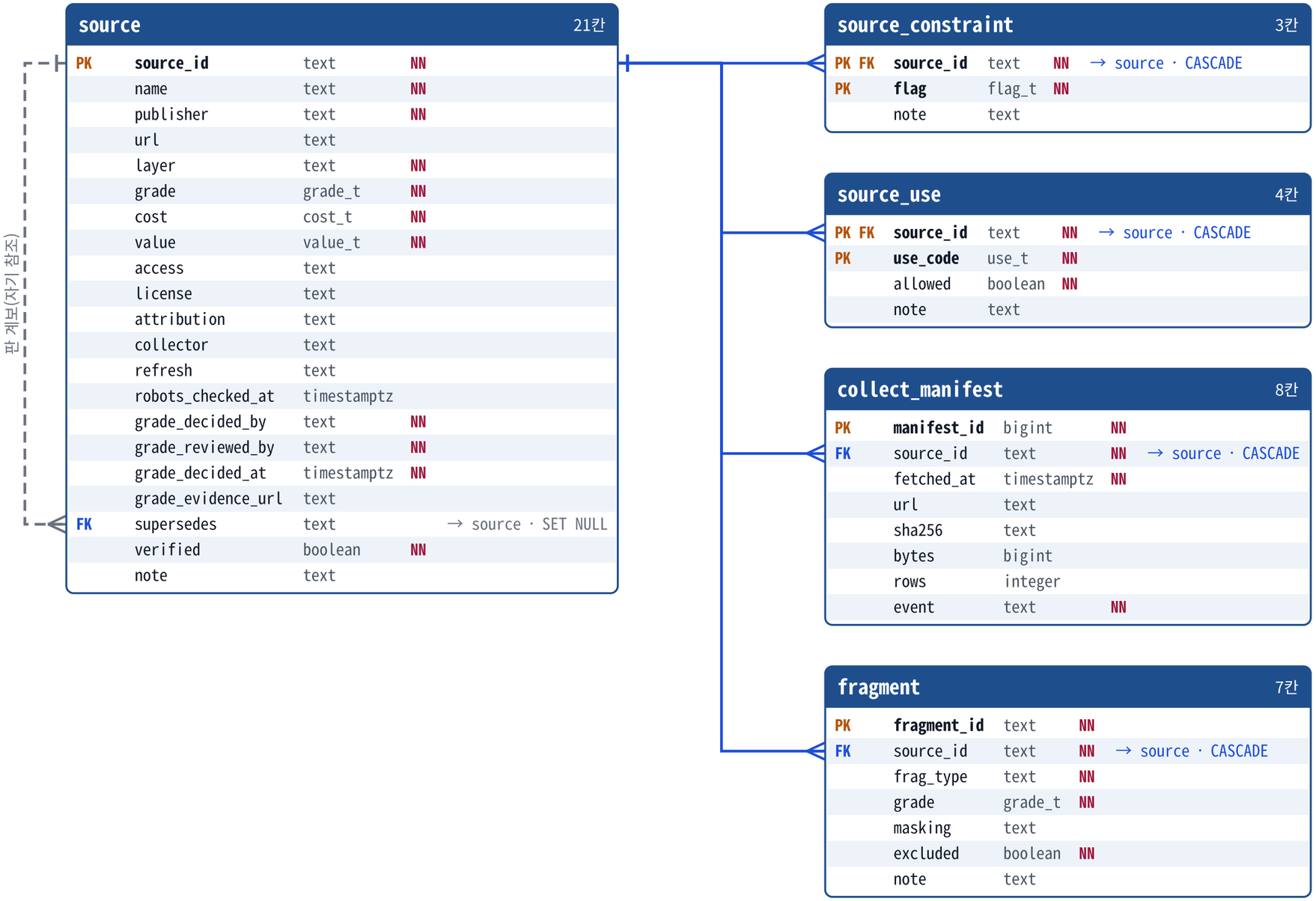


영역(상세 쪽): 원천 · 수집 · 원문 · 청크 · 임베딩 · 사전 · 제재 · 법령 · 골든 · 라벨 · 재현성 · 회원 · 검수 이력 · 생성 · 일괄 작업 · 운영 · 관리

ON DELETE CASCADE(부모를 지우면 함께 지워짐) · ON DELETE SET NULL(부모를 지우면 칸만 비움) · NO ACTION(참조하는 행이 있으면 부모 삭제가 막힘) · 다형 참조(DB 외래키 아님)

선은 부모(막대 ┤) → 자식(까마귀발 <) 방향이고 서로 엇갈리는 선은 없다(같은 부모의 갈래만 줄기를 같이 쓴다). 자기 참조 고리 둘: source.supersedes(SET NULL) · golden_sample.surface_of(CASCADE).
옅은 바탕 상자 = 영역(상세 쪽 하나). 어느 칸이 어느 칸을 가리키는지는 각 상세 쪽(선이 칸 줄에 닿음)과 §10 목록에 있다.

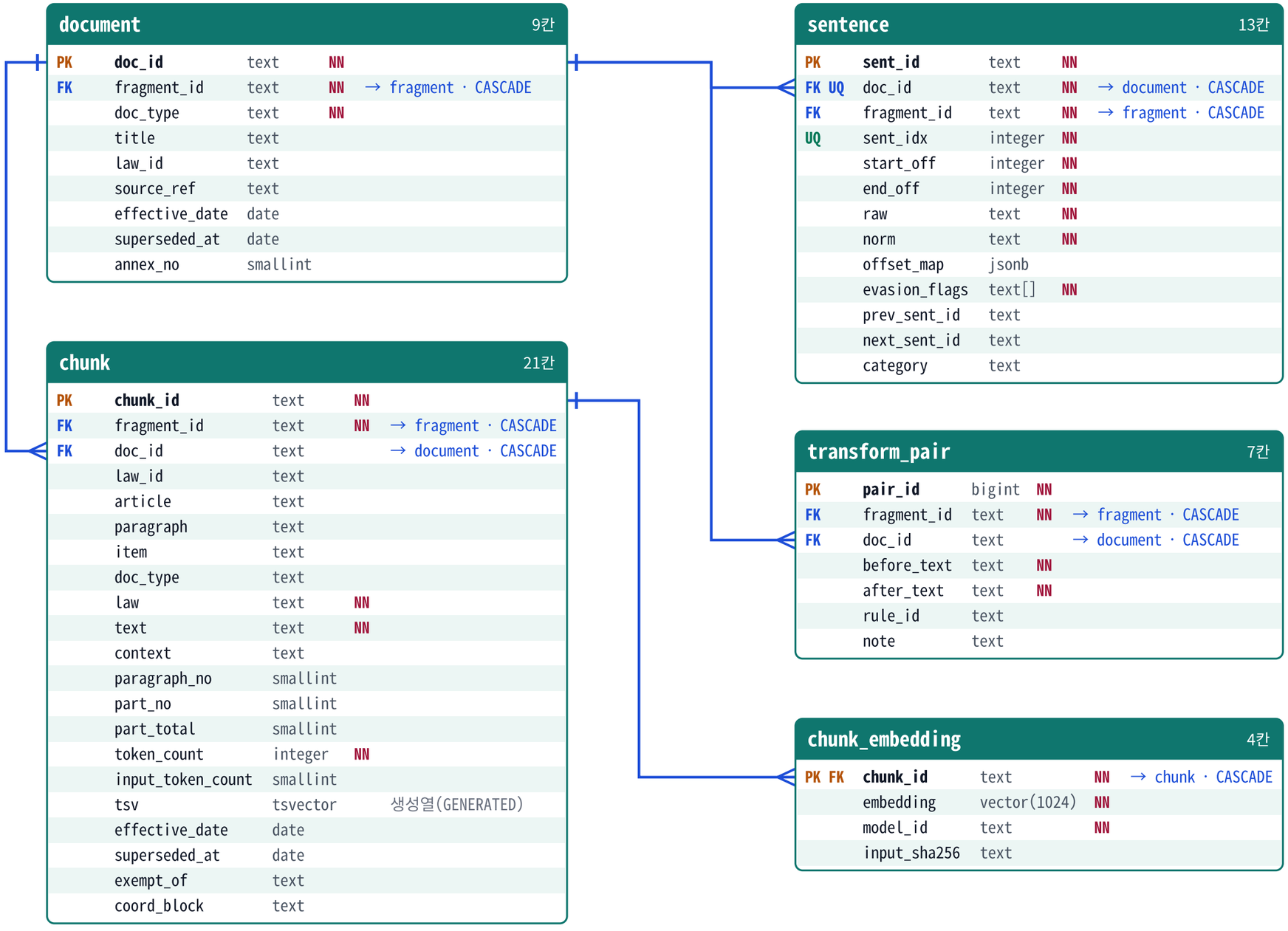
3. 원천 · 수집 — 표 5



fragment 아래의 파생 표 10개(문서 · 문장 · 청크 · 사전 · 제재 · 골든 등)는 다음 쪽들에 있다 — 각 표의 fragment_id 칸에 「→ fragment · CASCADE」로 적었다.

ON DELETE CASCADE(부모를 지우면 함께 지워짐)　　ON DELETE SET NULL(부모를 지우면 칸만 비움)

4. 원문 · 청크 · 임베딩 — 표 5



네 표의 fragment_id 는 「원천 · 수집」 쪽의 fragment 에 CASCADE 로 매달린다. golden_sample.sent_id 는 이 쪽의 sentence 를 가리킨다(「골든 · 라벨 · 재현성」 쪽).

UQ 묶음 — sentence 의 (doc_id, sent_idx) 가 함께 유일하다(uq_sentence_pos · 칸 하나씩 유일한 것이 아님).

ON DELETE CASCADE(부모를 지우면 함께 지워짐)

5. 사전 · 제재 · 법령 — 표 6

sanction_rule — 20칸

PK	**rule_id**	bigint	NN	
FK	fragment_id	text	NN	→ fragment · CASCADE
	law_id	text	NN	
	annex_no	text		
	violation_type	violation_t	NN	
	offense_count	smallint	NN	
	sanction_kind	text	NN	
	sanction_value	numeric		
	unit	text		
	risk_level	risk_t	NN	
	effective_date	date		
	superseded_at	date		
	verified_by	text		
	reviewed_by	text		
UQ	rule_key	text		
	annex1	text[]		
	cover	text		
	quote	text		
	fact_kind	text		
	plan_sha	text		

dict_entry — 9칸

PK	**entry_id**	bigint	NN	
FK	fragment_id	text	NN	→ fragment · CASCADE
UQ	dict_kind	text	NN	
UQ	term	text	NN	
	violation_type	violation_t		
	law_ref	text		
	category_scope	text[]	NN	
	exact_match	boolean	NN	
	confidence	numeric		

product_fact — 9칸

PK	**fact_id**	bigint	NN	
FK UQ	fragment_id	text	NN	→ fragment · CASCADE
UQ	ingredient	text	NN	
	recognition_no	text		
UQ	functional_claim	text	NN	
	daily_intake	text		
	caution	text		
	category	text	NN	
UQ	recog_kind	text	NN	

penalty_rule — 9칸

PK	**rule_id**	bigint	NN	
FK	fragment_id	text	NN	→ fragment · CASCADE
	tier	text	NN	
	base_rate	numeric		
	base_amount	numeric		
	aggravation	jsonb		
	mitigation	jsonb		
	effective_date	date	NN	
	superseded_at	date		

penal_clause — 9칸

PK	**clause_id**	bigint	NN	
FK	fragment_id	text	NN	→ fragment · CASCADE
	law_id	text	NN	
	article	text	NN	
	violation_type	violation_t		
	max_imprisonment_years	smallint		
	max_fine_krw	bigint		
	effective_date	date		
	superseded_at	date		

violation_article — 6칸

PK	**violation**	violation_t	NN
PK	**law_id**	text	NN
PK	**article**	text	NN
PK	**item**	text	NN
	adopted	boolean	NN
	note	text	

이 쪽 표 다섯은 모두 fragment(「원천 · 수집」 쪽)에 CASCADE 로 매달리고, 서로 사이에는 FK 가 없다. violation_article 은 FK 가 하나도 없는 대응표다.

UQ 묶음 — dict_entry (dict_kind, term) · product_fact (fragment_id, ingredient, functional_claim, recog_kind) 는 묶음으로 유일하다. sanction_rule.rule_key 는 부분 유일 인덱스(rule_key IS NOT NULL 인 행끼리 유일).

violation_article 의 PK 넷은 (violation, law_id, article, item) 한 묶음의 기본키다.

ON DELETE CASCADE(부모를 지우면 함께 지워짐)

6. 골든 · 라벨 · 재현성 — 표 3

golden_sample 24칸

PK	**sample_id**	text	NN	
FK	sent_id	text		→ sentence · CASCADE
FK	fragment_id	text	NN	→ fragment · CASCADE
	text	text	NN	
	category	text		
	violations	violation_t[]	NN	
	three_elem	jsonb		
	infeasibility	infeas_t		
	risk	risk_t		
	evidence	jsonb		
	claim_spans	jsonb		
	certainty	certainty_t		
	tense	tense_t		
	sent_type	senttype_t		
	origin	origin_t	NN	
	rule_id	text		
	surface_variant	surface_t	NN	
FK	surface_of	text		→ golden_sample · CASCADE
	provenance	text	NN	
	redistributable	boolean	NN	
	split	split_t	NN	
	unit	text	NN	
	cond	text		
	evidence_candidate	jsonb		

표면 변형 → 원문(자기 참조)

segment 8칸

PK	**segment_id**	text	NN
	label	text	
	member_count	integer	NN
	vulnerable_flag	boolean	NN
	top_terms	text[]	NN
	demographics	jsonb	
	source_set	text[]	NN
	built_at	timestamptz	NN

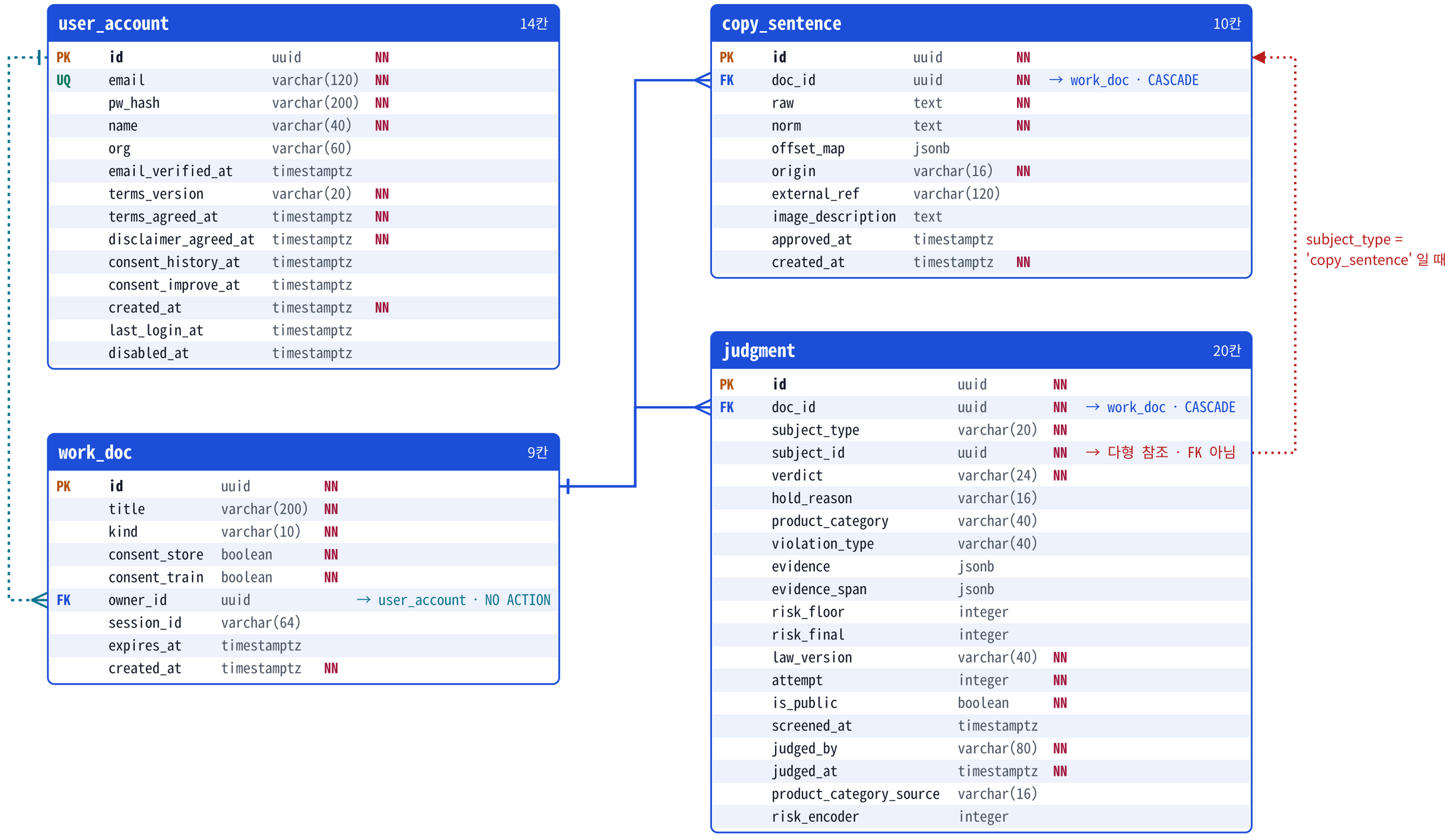
dataset_manifest 7칸

PK	**manifest_id**	text	NN
	source_set_version	text	NN
	seed	integer	NN
	built_at	timestamptz	NN
	counts	jsonb	NN
	unmeasurable	text[]	NN
	source_sets	jsonb	NN

golden_sample.sent_id → sentence(「원문 · 청크 · 임베딩」 쪽) · fragment_id → fragment(「원천 · 수집」 쪽). segment · dataset_manifest 는 FK 가 없다.

ON DELETE CASCADE(부모를 지우면 함께 지워짐)

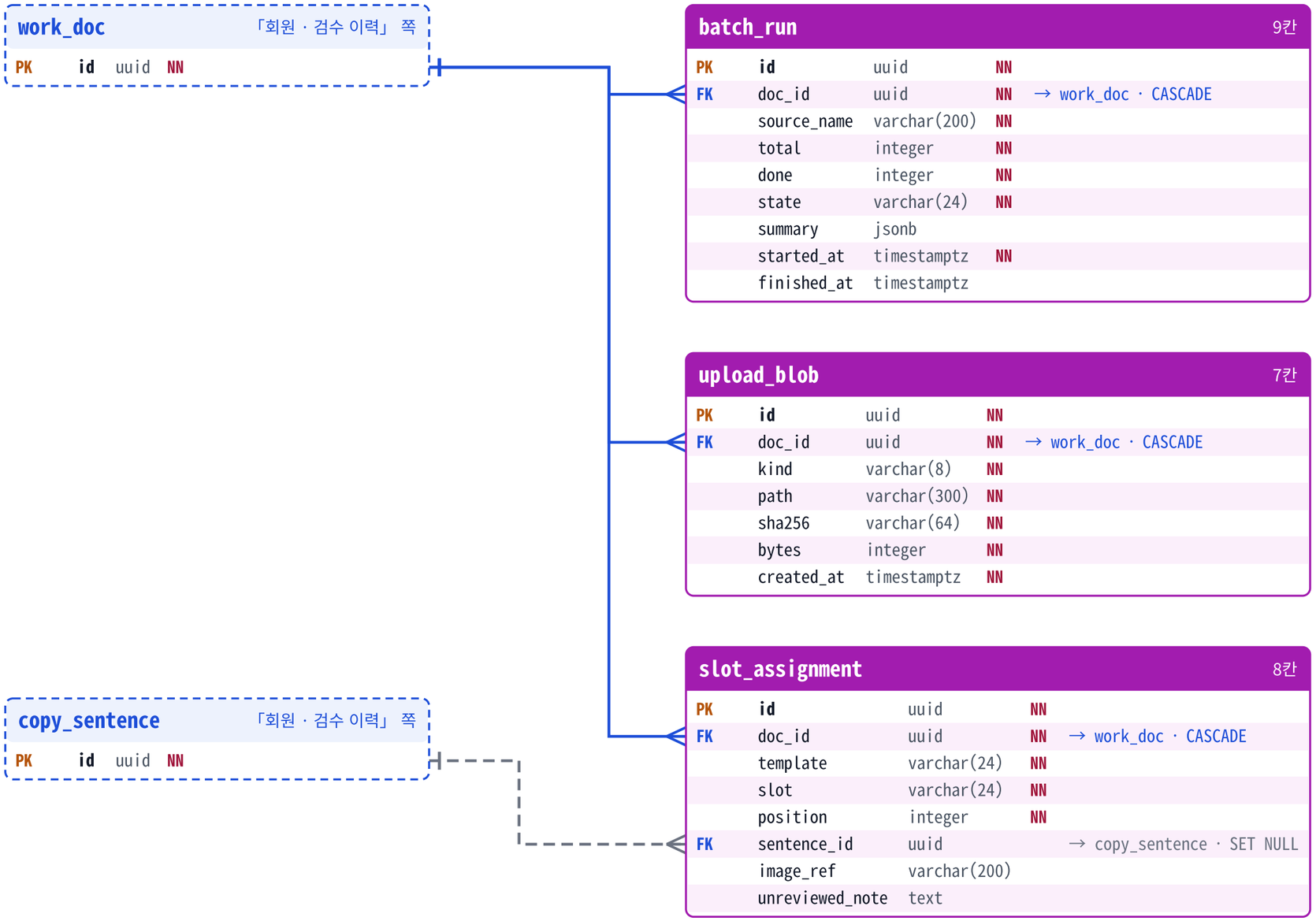
7. 회원 · 검수 이력 — 표 4



work_doc 이 하위 5표의 CASCADE 루트다(나머지 셋은 「생성 · 일괄 작업」 쪽). user_account.id 는 ticket.requester_id(「운영 · 관리」 쪽)도 가리킨다 — NO ACTION.

ON DELETE CASCADE(부모를 지우면 함께 지워짐)　　NO ACTION(참조하는 행이 있으면 부모 삭제가 막힘)　　다형 참조(DB 외래키 아님)

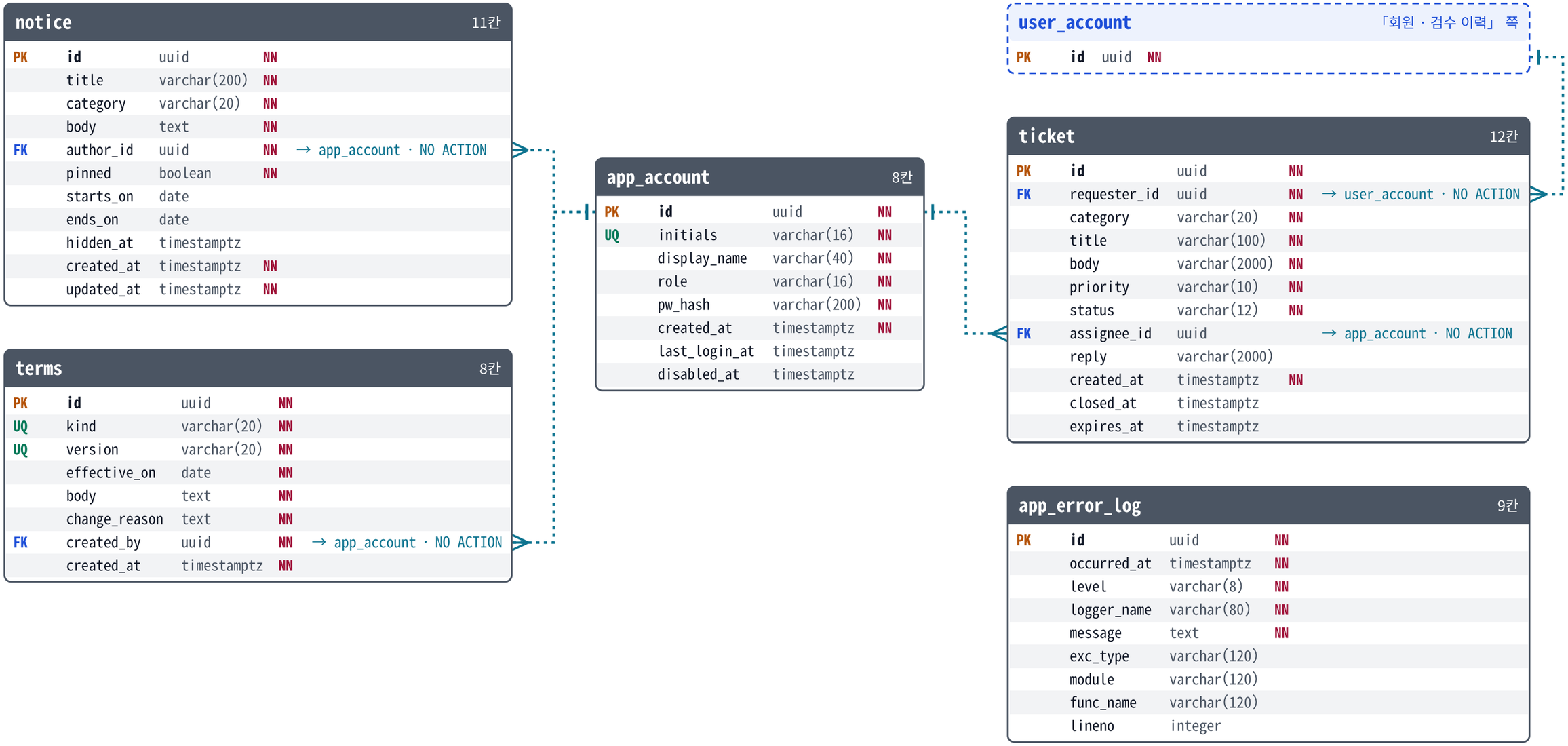
8. 생성 · 일괄 작업 — 표 3



점선 상자는 다른 쪽에 칸 전부가 있는 부모 표다(여기서는 기본키만). 이 쪽 세 표는 2026-10-21 기준 표만 있고 채우는 코드가 없다(02 문서 §6).

ON DELETE CASCADE(부모를 지우면 함께 지워짐) ON DELETE SET NULL(부모를 지우면 칸만 비움)

9. 운영 · 관리 — 표 5



계정 표를 가리키는 FK 다섯(이 쪽 넷 + work_doc.owner_id)은 모두 NO ACTION — 참조하는 행이 있으면 계정 행을 지울 수 없다. app_error_log 는 FK 가 없다.

UQ 묶음 — terms 의 (kind, version) 이 함께 유일하다(유일 인덱스 uq_terms_kind_version). app_account.initials · user_account.email 은 칸 하나로 유일.

NO ACTION(참조하는 행이 있으면 부모 삭제가 막힘)

10. 외래키 32개 목록

DB `pg_constraint` 의 외래키 전부다. 「그림」 칸은 그 FK 를 선으로 그린 절이다(다른 영역의 부모를 가리키는 FK 는 해당 절의 칸 오른쪽에 글자로 적었다).

#	자식 표 . 칸	부모 표 . 칸	ON DELETE	층	그림	제약 이름
1	`source.supersedes`	`source.source_id`	SET NULL	데이터	§2 · §3	`source_supersedes_fkey`
2	`source_constraint.source_id`	`source.source_id`	CASCADE	데이터	§2 · §3	`source_constraint_source_id_fkey`
3	`source_use.source_id`	`source.source_id`	CASCADE	데이터	§2 · §3	`source_use_source_id_fkey`
4	`collect_manifest.source_id`	`source.source_id`	CASCADE	데이터	§2 · §3	`collect_manifest_source_id_fkey`
5	`fragment.source_id`	`source.source_id`	CASCADE	데이터	§2 · §3	`fragment_source_id_fkey`
6	`document.fragment_id`	`fragment.fragment_id`	CASCADE	데이터	§2 (선) · §4 (글자)	`document_fragment_id_fkey`
7	`sentence.fragment_id`	`fragment.fragment_id`	CASCADE	데이터	§2 (선) · §4 (글자)	`sentence_fragment_id_fkey`
8	`transform_pair.fragment_id`	`fragment.fragment_id`	CASCADE	데이터	§2 (선) · §4 (글자)	`transform_pair_fragment_id_fkey`
9	`chunk.fragment_id`	`fragment.fragment_id`	CASCADE	데이터	§2 (선) · §4 (글자)	`chunk_fragment_id_fkey`
10	`sanction_rule.fragment_id`	`fragment.fragment_id`	CASCADE	데이터	§2 (선) · §5 (글자)	`sanction_rule_fragment_id_fkey`
11	`dict_entry.fragment_id`	`fragment.fragment_id`	CASCADE	데이터	§2 (선) · §5 (글자)	`dict_entry_fragment_id_fkey`
12	`product_fact.fragment_id`	`fragment.fragment_id`	CASCADE	데이터	§2 (선) · §5 (글자)	`product_fact_fragment_id_fkey`
13	`penalty_rule.fragment_id`	`fragment.fragment_id`	CASCADE	데이터	§2 (선) · §5 (글자)	`penalty_rule_fragment_id_fkey`
14	`penal_clause.fragment_id`	`fragment.fragment_id`	CASCADE	데이터	§2 (선) · §5 (글자)	`penal_clause_fragment_id_fkey`
15	`golden_sample.fragment_id`	`fragment.fragment_id`	CASCADE	데이터	§2 (선) · §6 (글자)	`golden_sample_fragment_id_fkey`
16	`sentence.doc_id`	`document.doc_id`	CASCADE	데이터	§2 · §4	`sentence_doc_id_fkey`
17	`transform_pair.doc_id`	`document.doc_id`	CASCADE	데이터	§2 · §4	`transform_pair_doc_id_fkey`
18	`chunk.doc_id`	`document.doc_id`	CASCADE	데이터	§2 · §4	`chunk_doc_id_fkey`
19	`chunk_embedding.chunk_id`	`chunk.chunk_id`	CASCADE	데이터	§2 · §4	`chunk_embedding_chunk_id_fkey`
20	`golden_sample.sent_id`	`sentence.sent_id`	CASCADE	데이터	§2 (선) · §6 (글자)	`golden_sample_sent_id_fkey`
21	`golden_sample.surface_of`	`golden_sample.sample_id`	CASCADE	데이터	§2 · §6	`golden_sample_surface_of_fkey`
22	`copy_sentence.doc_id`	`work_doc.id`	CASCADE	런타임	§2 · §7	`copy_sentence_doc_id_fkey`
23	`judgment.doc_id`	`work_doc.id`	CASCADE	런타임	§2 · §7	`judgment_doc_id_fkey`
24	`batch_run.doc_id`	`work_doc.id`	CASCADE	런타임	§2 · §8	`batch_run_doc_id_fkey`
25	`upload_blob.doc_id`	`work_doc.id`	CASCADE	런타임	§2 · §8	`upload_blob_doc_id_fkey`
26	`slot_assignment.doc_id`	`work_doc.id`	CASCADE	런타임	§2 · §8	`slot_assignment_doc_id_fkey`
27	`slot_assignment.sentence_id`	`copy_sentence.id`	SET NULL	런타임	§2 · §8	`slot_assignment_sentence_id_fkey`

#	자식 표 . 칸	부모 표 . 칸	ON DELETE	층	그림	제약 이름
28	work_doc.owner_id	user_account.id	NO ACTION	런타임	§2 · §7	fk_work_doc_owner_id_user_account
29	ticket.requester_id	user_account.id	NO ACTION	런타임	§2 · §9	ticket_requester_id_fkey
30	notice.author_id	app_account.id	NO ACTION	런타임	§2 · §9	notice_author_id_fkey
31	terms.created_by	app_account.id	NO ACTION	런타임	§2 · §9	terms_created_by_fkey
32	ticket.assignee_id	app_account.id	NO ACTION	런타임	§2 · §9	ticket_assignee_id_fkey

합계 32 — CASCADE 25 · SET NULL 2 · NO ACTION 5 · 데이터 층 21 · 런타임 층 11. 모든 FK 는 칸 하나짜리이고 부모의 기본키를 가리킨다. 자기 참조 2개(#1 source.supersedes · #21 golden_sample.surface_of). 판정의 subject_id 는 다형 참조라 이 목록에 없다(§ 7).

삭제 규칙이 실제로 걸리는 곳 — ksr b54246d 코드 기준

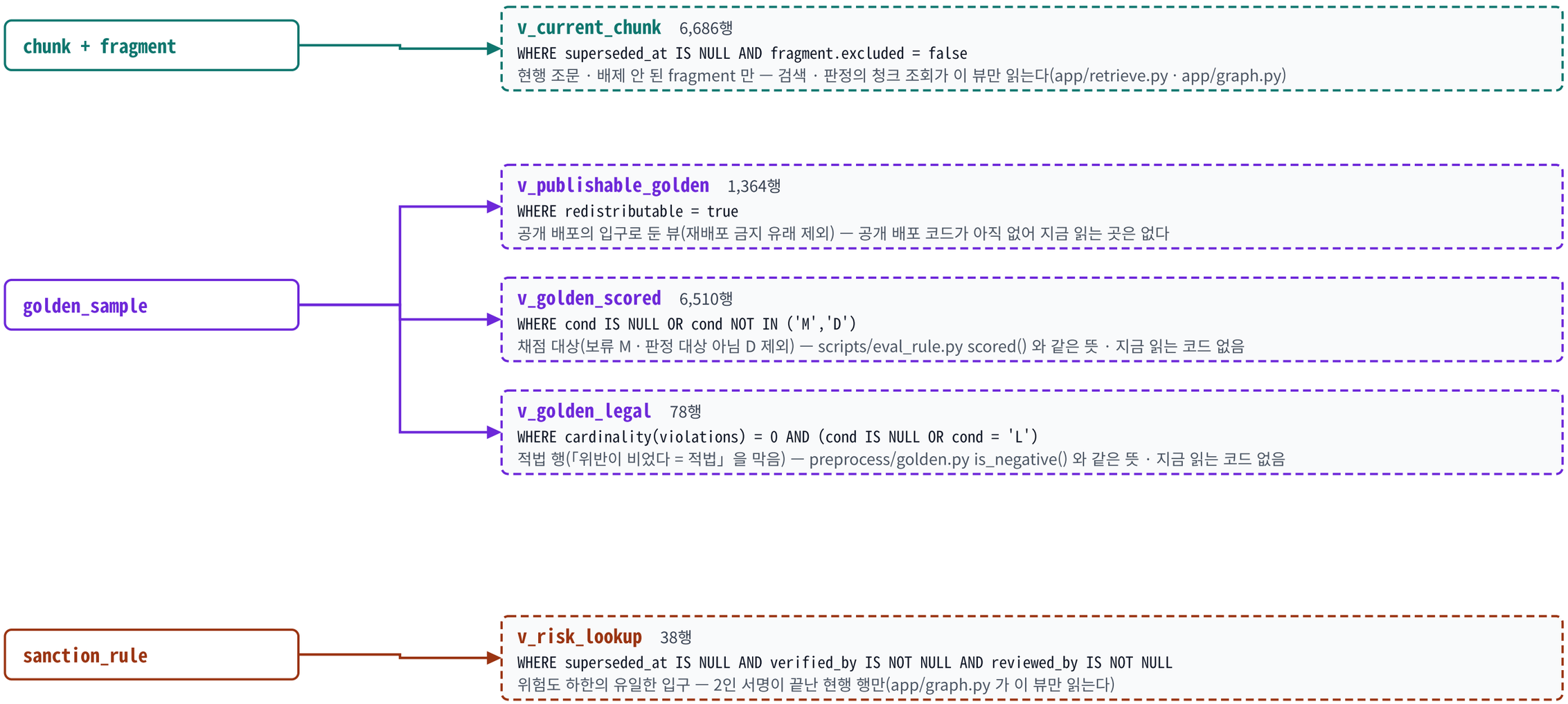
ON DELETE 규칙은 「부모 행이 지워지면」 일어나는 일이다. 코드에서 행을 지우는 곳은 아래가 전부다(DELETE · delete(를 app/ · scripts/ · preprocess/ · collect/ 에서 찾음 · 시험 코드 제외). DB 전체를 지우고 다시 세우는 개발 도구 scripts/db_reset.py (도커 볼륨 삭제)는 행 단위 삭제가 아니어서 표에 넣지 않았다.

지우는 코드	지우는 행	규칙 때문에 함께 바뀌는 것
app/judgment_store.py replace_review	품목을 고른 재판정을 적을 때 — 같은 회원 · 최근 6시간 · 문장 묶음이 같은 가장 최근 work_doc 1행	copy_sentence · judgment (doc_id CASCADE)가 함께 지워지고, 새 판정이 새 문서로 다시 적힌다. batch_run · upload_blob · slot_assignment 도 CASCADE 자식이지만 지금 0행이다
scripts/load_db.py 소스 거두기	레지스트리(data_sources.yaml)에서 빠진 source — 단 fragment · collect_manifest · 자기 참조(supersedes)가 하나라도 남아 있으면 지우지 않고 남긴다	source_constraint · source_use (CASCADE). 이 경로로는 fragment 아래 파생물까지 내려가는 삭제가 일어나지 않는다
scripts/load_db.py 골든 거두기	선언(골든 파일)에 없는 golden_sample	그 행을 원문으로 둔 표면 변형 행(surface_of CASCADE)
scripts/embed.py 청크 거두기	선언(chunks.jsonl)에 없는 chunk	chunk_embedding (CASCADE)
scripts/load_db.py 사전 · 제품 사실 · 제재 · 대응표	원천에서 빠진 dict_entry · product_fact · sanction_rule 행 · 원천에서 빠진 source_constraint · source_use 행 · violation_article 전체(지우고 다시 넣음)	없음(이 표들은 자식 표가 없다)
app/error_log.py	보관 90일(ERROR_LOG_RETENTION_DAYS)이 지난 app_error_log — 기록기가 줄을 쓸 때 6시간에 한 번	없음(FK 없음)

- 계정 표(user_account · app_account)를 지우는 코드는 없다. 탈퇴는 user_account.disabled_at 기록이고, 계정을 가리키는 FK 다섯은 NO ACTION 이라 참조하는 행이 남아 있으면 DB 가 계정 삭제를 막는다.
- work_doc.expires_at (비회원 문서 · 동의 없는 문서) · ticket.expires_at (닫힌 뒤 180일)은 지울 기한을 담는 칸이다. 그 기한으로 행을 지우는 코드는 b54246d 에 없다.
- judgment.subject_id 는 FK 가 아니어서 copy_sentence 한 행만 지우면 판정이 남는다. 지금 코드는 문장을 따로 지우지 않고 늘 work_doc 단위로 지운다(위 첫 줄).
- 다시 판정한 이력의 정책 — 결정기록은 「고른 재판정은 앞 기록을 지우지 않고 새 판정 행으로 쌓고, 품목 출처를 user_selected 로 남기며, 홈 통계는 같은 사람 · 같은 문구의 가장 최근 판정만 센다」로 정했다(D-348 보충 36 결정 2 · 2026-10-21). 코드 반영 전이라 지금은 위 첫 줄처럼 지우고 다시 적는다. 이 결정은 스키마를 바꾸지 않는다(02 문서 § 6.3 · § 13 과 같은 서술).

11. 뷰 5개

뷰는 행을 따로 갖지 않는 저장된 SELECT 다. 잊기 쉬운 필터(현행만 · 2인 서명 끝난 것만 · 재배포 가능한 것만 등)를 DB 안에 고정해 둔다. 지금 코드가 원표 대신 읽는 뷰는 둘이다 — 검색 · 판정의 청크 조회는 `v_current_chunk` 만(`app/retrieve.py` · `app/graph.py`), 위험도 하한은 `v_risk_lookup` 만(`app/graph.py`) 읽는다. 골든 뷰 셋은 지금 읽는 코드가 없다 — 평가 스크립트는 골든을 파일(`golden.jsonl`)에서 읽고, `v_golden_scored` · `v_golden_legal` 은 같은 뜻의 파이썬 판정(`eval_rule.scored` · `golden.is_negative`)과 조건이 같은지 게이트 시험이 맞춰 본다. 조건은 DB `pg_views` 의 정의를 줄여 옮겼다.



점선 상자 = 뷰(행을 따로 갖지 않는 저장된 SELECT). 행 수는 같은 조회 시각의 이 PC DB 값.
골든 뷰 셋은 같은 뜻의 파이썬 판정과 조건이 같은지 게이트 시험이 맞춰 본다(tests/test_guide_golden.py · v_golden_scored · v_golden_legal). 평가 스크립트(eval_rule · eval_graph)는 골든을 DB 가 아니라 golden.jsonl 에서 읽는다.

정의 위치 — `db/schema.sql`(5개 모두) · `db/migrations/0013_risk_lookup_four_eyes.sql`(`v_risk_lookup` 의 2인 서명 조건) · `db/migrations/0024_golden_cond.sql`(`v_golden_scored` · `v_golden_legal`). `v_current_chunk` 는 chunk 의 칸을 그대로 내므로 chunk 에 칸을 더하면 뷰를 다시 만들어야 한다(DB 뷰 주석).

12. 근거와 확인

확인한 것	방법	결과
표 · 칸 · 타입 · NOT NULL · PK · FK · UQ	`catalog.py` 가 DB 카탈로그를 읽기 전용 트랜잭션으로 SELECT → `DB카탈로그_15.json` → `make_erd15.py` 가 그림을 그린다(손으로 적은 칸 없음)	표 31 · 칸 323 · FK 32
DB 와 `app/models.py`	`ksr b54246d` 의 ORM 메타데이터 12표의 칸 이름 · NULL 허용을 DB 와 대조	다른 곳 0
DB 와 `db/schema.sql`	`ksr b54246d` 의 `CREATE TABLE` 19개의 칸 이름을 DB 와 대조 · REFERENCES 21개 = DB 데이터 층 FK 21개	다른 곳 0
02 문서 ERD 와	02 그림 1 · 2(`erd_gov.svg` · `erd_runtime.svg`)와 §4.1 FK 표의 표 · 관계를 이 문서와 대조	표 31 · FK 32 · 삭제 규칙 같음
독립 재조회(검수 R1)	2026-10-21 18:11 KST 에 `information_schema`(columns · table_constraints · key_column_usage · referential_constraints · views)로 새로 SELECT 해 그림 SVG 의 글자 · §10 표 · 부록 행 수와 한 칸씩 대조	표 31 · 칸 323 · FK 32 · 뷰 5 · 행 수 모두 같음
서술 · 가독성(검수 R2)	표별 설명 · 뷰 설명 · 삭제 규칙 설명을 `git show b54246d:` 코드로 대조(`app/judgment_store.py` · `app/graph.py` · `app/retrieve.py` · `app/claim_match.py` · `collect/registry.py` · `scripts/load_db.py` · `scripts/embed.py` · `app/error_log.py`) · 모든 쪽을 96 dpi(A3 실제 크기) · 150 dpi 로 떠서 선을 하나씩 따라감	표별 설명 2곳 · §11 뷰 설명 고침 · 삭제 규칙이 실제로 걸리는 곳 표 추가(§10) · 개요(§2)와 §9 의 선 배치 고침
최종 형태 · 표본(검수 R3)	2026-10-22 08:46 KST 에 `information_schema` 를 다시 SELECT 해 무작위 표 5개(`source` · `golden_sample` · `segment` · `document` · `upload_blob`)의 칸 69개 전부와 FK 10개를 PDF 글자와 대조 · 31표 · 뷰 5 행 수 재조회 · 모든 쪽을 96 · 150 dpi 로 떠서 확인	다른 곳 0 · 행 수 같음
기준 커밋 뒤 변경	`git diff b54246d HEAD -- app/models.py db/ alembic/`	변경 없음

- 이 PC 의 DB 는 정본 기기가 아니라 개발용 복제본(`DATA_ROLE=replica`)이다. 스키마(표 · 칸 · 제약)는 저장소 head 와 같은 `0027` 판이고, 행 수만 기기마다 다를 수 있다.
- 칸의 뜻 · CHECK 제약 · 인덱스 · 설계 이유는 02 문서 §5 · §6 · §9 · §10 에 있다. 이 문서는 표와 관계를 보는 그림에 집중한다.
- 2주차 ERD 에 있던 「왜 두 층인가」(§0) · 스키마가 강제하는 규칙(§1-2) · 런타임 층 설계 판단(§2-1) · VectorDB 구조(§3)는 현행 값으로 02 문서 §2 · §5 · §6 · §1.2 · §10 과 08 벡터DB / GraphDB 구축 결과서가 이어 받아 이 문서에는 다시 싣지 않았다. 「두 층을 잇는 지점」(§4)은 이 문서 §11 의 두 뷰(판정 · 검색이 데이터 층을 읽는 자리)와 §7 의 `work_doc` · `copy_sentence` · `judgment`(판정 결과를 적는 표)로 본다.

부록 — 표별 한 줄 설명

행 수는 2026-10-21 17:52 KST 이 PC DB 의 `count(*)` 다. 데이터 층의 행 수는 02 문서(같은 날 15:44 조회)와 같다. 런타임 층의 행 수는 시험 사용으로 늘어나는 값이라 02 문서와 다를 수 있고, 서비스 이용 수치로 읽지 않는다.

#	표	층	절	칸	행	한 줄 설명
1	`source`	데이터	§3	21	53	소스 레지스트리 — `data_sources.yaml` 의 DB 표현 · 등급 · 2인 확인 서명(`ck_source_four_eyes`)
2	`source_constraint`	데이터	§3	3	58	소스별 제약 플래그(`flag_t` 12종 · NOREDIST · NOTRAIN 등) — 소스 1 : 플래그 N
3	`source_use`	데이터	§3	4	212	소스 × 용도(U1 학습 ~ U4 배포) 허용 여부 — 레지스트리의 DB 사본. 검색(`U2_rag`) · 인용(`U3_cite`) 질의가 읽는다(`app/retrieve.py`)
4	`collect_manifest`	데이터	§3	8	19,122	수집 이력 — url · sha256 · bytes · rows · event(같은 sha256 은 다시 넣지 않음)
5	`fragment`	데이터	§3	7	21	추출 단위 — 등급(grade)이 갈리는 곳 · 모든 파생 표가 `fragment_id` 로 매달리는 축
6	`document`	데이터	§4	9	62	문서 단위(법령 · 행정규칙 · 별표) — `superseded_at IS NULL` 이 현행
7	`sentence`	데이터	§4	13	0	원문 문장 — raw / norm / offset_map. 지금 적재 단계가 없어 비어 있다
8	`transform_pair`	데이터	§4	7	0	시정 전후 문구 쌍 — 적재기 없음
9	`chunk`	데이터	§4	21	6,686	조 · 항 · 호 단위 검색 청크(토큰 ≤ 512 · 4법) · 어휘 검색용 생성열 `tsv`
10	`chunk_embedding`	데이터	§4	4	6,686	청크 벡터 `vector(1024)` — 청크를 지우면 함께 지워진다
11	`sanction_rule`	데이터	§5	20	38	행정처분 기준 [별표] — 위반 유형 × 차수 → 처분 · 위험도(2인 서명 · 판정은 뷰 `v_risk_lookup` 으로만 읽음)
12	`dict_entry`	데이터	§5	9	4,342	금지 · 적법 표현 사전 — 정확 매칭(`exact_match`)만 위험도 하한 자격
13	`product_fact`	데이터	§5	9	447	인정 기능성 문구(정본 축만) — 판정 코드는 이 표를 읽지 않는다(기능성 문구 대조 `app/claim_match.py` 는 이 표의 원천 `hf_api_labels.jsonl` 과 고시형 `hf_gosi_claims.jsonl` · `mfds_hf_labels.jsonl` 파일을 직접 읽음)
14	`penalty_rule`	데이터	§5	9	0	과징금 고시 — 부과기준율 · 가중 · 감경. 적재기 없음
15	`penal_clause`	데이터	§5	9	0	벌칙 — 징역 · 벌금 상한. 적재기 없음
16	`violation_article`	데이터	§5	6	14	위반 유형 → 조문 대응표 — 타입이 아니라 데이터 · FK 없음
17	`golden_sample`	데이터	§6	24	8,383	골든셋 — 위반 라벨 · 측정 축 · 출처 · 분할 · 조건(이 스키마에서 칸이 가장 많다)
18	`segment`	데이터	§6	8	0	세그먼트 — `member_count ≥ 20` (k-익명) · 채우는 코드 없음
19	`dataset_manifest`	데이터	§6	7	0	데이터셋 판 · 재학습 기준 — 채우는 코드 없음
20	`user_account`	런타임	§7	14	7	일반 회원 — 탈퇴는 행 삭제가 아니라 `disabled_at` 기록
21	`work_doc`	런타임	§7	9	51	사용자 작업 문서 — 하위 5표의 CASCADE 루트 · 보관 · 학습 동의 · 동의 없으면 만료 시각 필수
22	`copy_sentence`	런타임	§7	10	203	사용자가 쓰거나 생성한 문구 한 줄 — raw / norm / offset_map · origin
23	`judgment`	런타임	§7	20	203	판정 — 다형 참조(subject_type · subject_id) · 판정 4값 · 위험도 칸 · CHECK 16개
24	`batch_run`	런타임	§8	9	0	배치 검수 1회(CSV 일괄) — 표만 있음
25	`upload_blob`	런타임	§8	7	0	사용자 업로드물(CSV · 이미지) 경로 — 표만 있음
26	`slot_assignment`	런타임	§8	8	0	템플릿 조립(AI 광고 생성) 자리 — 표만 있음 · 문장이 지워지면 `sentence_id` 만 비움

#	표	층	절	칸	행	한 줄 설명
27	`app_account`	런타임	§9	8	0	관리자 콘솔 계정(governor) — 가입 화면 없음
28	`notice`	런타임	§9	11	0	공지 — 분류(rule_change · system · update) · 게시 기간
29	`terms`	런타임	§9	8	0	약관 판 — 종류(service · privacy · disclaimer) × 판 유일
30	`ticket`	런타임	§9	12	2	문의 — 판정 원문 · 판정 id 를 담는 칸이 없다 · 닫히면 `expires_at` 이 파기 기한
31	`app_error_log`	런타임	§9	9	0	WARNING 이상 앱 오류 로그 — 마스킹 뒤 값만 · 보관 90일 · FK 없음